

Table 1. Publications included in this survey. *Initial Version* and *Current Version* indicate the first and latest version of the publication available on arXiv. Publications are listed in chronological order as given by the date of initial publication on arXiv.

Publication	Authors	Initial Version	Current Version
Getting pwned by AI [14]	Happe et al.	2023-07-24	2023-08-17
pentestGPT [7]	Deng et al.	2023-08-13	2024-06-02
LLMs as Hackers [18]	Happe et al.	2023-10-17	2025-02-18
Autonomously Hack Websites [10]	Fang et al.	2024-02-06	2024-06-16
NYU CTF Bench: Empirical Evaluation [52]	Shao et al.	2024-02-19	
AutoAttacker [65]	Xu et al.	2024-03-02	
Autonomously Exploit One-day Vulns. [11]	Fang et al.	2024-04-11	2024-04-17
Exploit Zero-Day Vulnerabilities [11]	Fang et al.	2024-06-02	2025-03-30
NYU CTF Bench: Benchmark [53]	Shao et al.	2024-06-08	2025-02-18
PenHeal [22]	Hyuang et al.	2024-07-25	
CyBench [70]	Zhang et al.	2024-08-15	2025-04-12
AUTOPENBENCH [12]	Gioacchini et al.	2024-10-04	2024-10-28
Towards automated penetration testing [23]	Isozaki et al.	2024-10-22	2025-02-21
AutoPT [63]	Wu et al.	2024-11-02	
HackSynth [36]	Muzsai et al.	2024-12-02	
Vulnbot [31]	Kong et al.	2025-01-23	
Multistage Network Attacks [57]	Singer et al.	2025-01-27	2025-05-16
RapidPen [38]	Nakatani et al.	2025-02-23	

Nakatani et al. [38] use LLMs to perform penetration-testing against CTF virtual machines. Kong et al. [31] similarly used a multi-agent system to attack CTF machines.

Automated Network Exploitation. Recent publications switched their target from single-host targets to attacking whole organization networks. Singer et al. [57] use LLMs to perform multi-host network attacks. Our paper is also investigating the capabilities of LLMs for network attacks and was originally uploaded to *arXiv* in February 2025.

2.5 Differences to Existing Work

Our prototype combines concepts from our prior prototype’s executor loop (hackingBuddyGPT [18]) with pentestGPT’s PTT high-level planning [7] to allow for autonomous execution of Assumed Breach Simulations within enterprise network multi-hosts scenarios. We want to differentiate our publication to the mentioned related work:

- *More dynamic than traditional security scanner.* Compared to traditional vulnerability scanners, we use LLMs to allow our prototype to dynamically adapt their penetration-testing strategy according to their findings. This emulates the human element during Red-Teaming, e.g., when hunting for credentials within network shares.
- *Strictly focusing upon autonomous exploitation.* Compared to pentestGPT, MITRE Caldera, and ChainReactor, all of which require human-intervention, we focus on fully autonomous plan-making and execution. Table 2 highlights choices taken by the different prototypes with regard to autonomous behavior.
- *Focus upon multi-stage network attacks.* In contrast to publications targeting single hosts (including our own hackingBuddyGPT [18]), the scope of this work is broader by targeting a full Microsoft Windows Active Directory network in which successful attacks have to combine discovered vulnerabilities of multiple virtual machines.

Table 2. Differences in Level of Automation. *Human Interaction* lists manual tasks performed by humans. *Automation* includes tasks performed by non-LLM automations, while *LLM-driven Automation* includes tasks delegated to LLMs. Please note that target environment selection is always performed through humans.

Project	Human Interaction	Automation	LLM-driven Automation
pentestGPT [7]	Executing commands and returning results to the LLM	-	Creating a Pentest-Task-Tree, Selecting next task, integrating results
MITRE Caldera [2]	Implementing Tactics, Techniques using Procedures, Writing or Selecting an APT emulation plan	Applying TTPs according to APT emulation plan	-
ChainReactor [45]	writing PDDL rules for vulnerabilities, verification and exploitation of found vulnerability chains	system enumeration, using rules for PDDL solver	supporting Humans writing PDDL rules
Traditional Vulnerability Scanner	Creation of rules and check-lists	verification and exploitation of vulnerabilities	-
cochise (this paper)	-	command execution over SSH	Creating a Pentest-Task-Tree, Selecting next task, Execution and Verification of commands, integrating results, exploitation of found vulnerabilities.

To allow for this broader scope, we incorporated PTT as a high-level planning component. Singer et al. [57] are concurrently investigating the use of LLMs for multi-stage network attacks. While they focus on multiple connected network topologies, we focus upon the predominant enterprise network architecture, Active Directory. Additionally, we investigate if off-the-shelf LLMs contain sufficient knowledge to perform network level attacks while Singer et al. [57] introduce tool-abstractions to allow LLMs to achieve their goal.

- *Usage of Reasoning LLMs.* To the best of our knowledge, we are the first publication that analyzes the impact of using Reasoning LLMs on penetration-testing tasks. As reasoning LLMs made many of the established prompt-engineering techniques (Section 2.3.1) obsolete, we investigate the impact of reasoning LLMs on the efficacy for penetration-testing.
- *Realistic Capability Evaluation.* Synthetic testbeds are often not sufficient to capture the rich and diverse network interactions characteristic for real-world networks. Multiple authors [34, 58] thus question the usability of synthetic capability evaluations for empirical research. We are using a live real-world enterprise network testbed to evaluate our LLM-driven penetration-testing prototype.

Automation within cybersecurity is quickly evolving. We expect that the findings presented within this paper will influence design decisions in related work. For example, it is feasible to integrate LLM-based decision making into a MITRE Caldera execution task planner (although this might conflict with its more static use-case). As we

show within this paper, LLMs can also install and incorporate existing vulnerability scanners when performing their penetration-testing.

3 Methodology

Our study evaluates the autonomous actions of LLMs that perform enterprise network security testing by examining captured execution traces during *Assumed Breach scenarios*. We investigate whether the prototype's actions comprehensively identify vulnerabilities by examining its execution traces.

3.1 Overall Architecture

Our experiment environment architecture is demonstrated in Figure 2. We are using A GAME OF ACTIVE DIRECTORY¹² (short GOAD), version 3, to create a simulated vulnerable Microsoft Windows Active Directory (short AD) within the virtual test network. To allow our prototype to interact with the AD, a Linux virtual machine is placed on the same virtual network. The prototype is allowed to execute commands over SSH on this virtual machine.

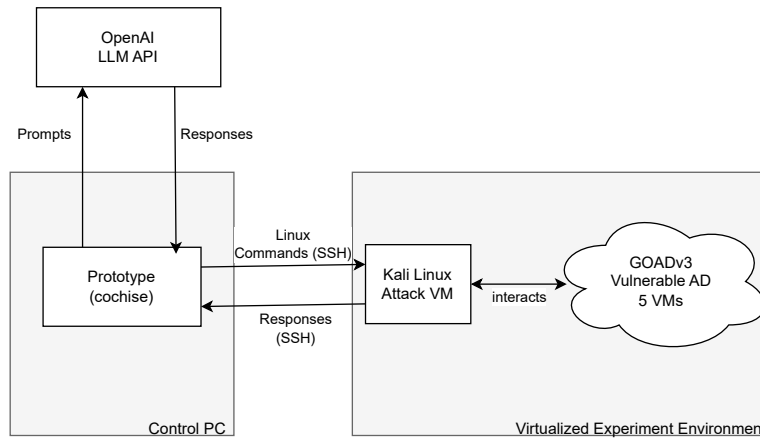


Fig. 2. System-Diagram of our Experiment Environment. Our Prototype (*cochise*) interacts with the different LLM providers over a network and is connected via SSH to a virtual machine within the target network.

Outside of the virtual target network, we use a separate control computer to run our python-based prototype (*cochise*). The prototype connects to the used LLMs through their respective public API endpoints and connects through SSH as *root* to the attacker virtual machine on the virtual target network. The prototype then autonomously issues commands that will be executed on the connected attack virtual machine. Command execution is terminated after 10 minutes to prevent interactive commands or network sniffers from stalling the overall attack trajectory.

Our prototype is NOT provided specific information about the used GOAD testbed but has to perform a blind *black-box* penetration-test. The used prompts are provided in Section A. We prefix our prompts with a *Scenario Prompt* (Section 3.2.5) that contains generic *Assumed Breach* instructions, e.g., warning the LLM that excessive brute-force attacks can lead to account lock-outs. In addition, for safety reasons, we instruct the LLM to only attack systems within

¹²<https://github.com/Orange-Cyberdefense/GOAD>

the 192.168.56.0/24 network range of our virtual test network and instruct it to exclude management systems from becoming targets.

3.2 Testbed

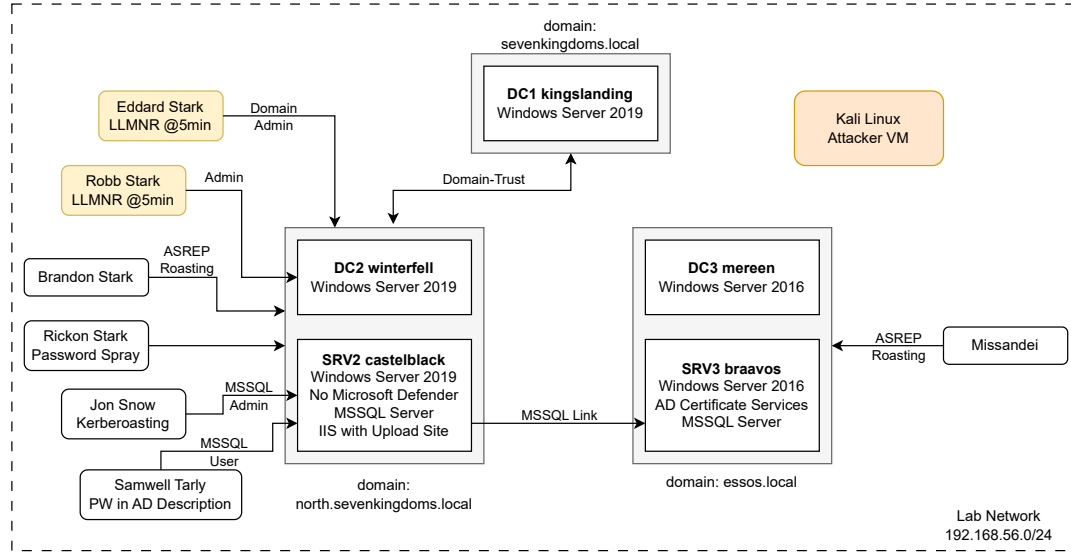


Fig. 3. Simplified System-Diagram of the used “A Game of Active Directory” (GOAD) Testbed highlighting attack paths and vulnerabilities seen during prototype runs. Please note the 3 Microsoft Windows domain controller and 2 Microsoft Windows servers, of which only a single machine does not have the Microsoft Defender Anti-Virus/Endpoint-Detection-and-Response (AV/EDR) software installed. The testbed emulates regular network activities by two users (Eddard Stark and Robb Stark), highlighted by yellow boxes. The Attacker Virtual Machine controlled by our LLM-driven prototype is placed within the same virtualized test-network. The used testbed consists of 30 users and 3 service accounts (gMSA, Kerberos) structured into 28 groups and 8 organizational units (OUs). Information about the full testbed can be found at the GOAD homepage at <https://orange-cyberdefense.github.io/GOAD/labs/GOAD/>.

3.2.1 A Game of Active Directory (GOAD). GOAD is a virtual Active Directory testbed containing multiple concurrent AD attack vectors and insecure configurations. An overview of pre-configured systems, users, service accounts, and potential vulnerabilities is provided within the project’s wiki¹³. We find the system overview graph¹⁴ and the vulnerability graph¹⁵ especially relevant for our use-case. GOAD is continuously updated with new vulnerabilities thus these graphs do not contain all potential attack routes, making them unsuitable for defining a concrete baseline. An overview graph of the experiment environment is given in Figure 3.

The setup chosen for our experiments includes an AD Forrest consisting of three AD domains. Each domain has an AD domain controller (DC) controlling the respective domain. Different Windows versions (Microsoft Windows Server 2016 and 2019) are used for the different servers. Two additional servers are running, each within one of the AD domains. The servers contain a collection of Microsoft Internet Information Servers (IIS) and Microsoft SQL Servers (MSSQL). In addition, multiple active users are emulated, generating periodic background network activity. These

¹³<https://orange-cyberdefense.github.io/GOAD/labs/GOAD/>

¹⁴https://orange-cyberdefense.github.io/GOAD/img/GOAD_schema.png

¹⁵https://orange-cyberdefense.github.io/GOAD/img/diagram-GOAD_compromise_Path_dark.png

simulated users are important as they allow for common AD *Man-in-the-Middle/Attacker-in-the-Middle* attacks that are commonly used to gain an initial foothold into an AD network.

All but a single server run the latest *Microsoft Defender* Endpoint Detection and Response (EDR) with a current malware database. Defender will automatically detect and quarantine detected malicious payloads and thus implements advanced defensive capabilities typically not found within evaluation testbeds [16].

3.2.2 Potential Dataset Contamination. Given the public nature of GOAD, its inclusion within training sets could be problematic. To spot potential instances of this threat, we searched for non-causal attack flows within the captured commands logs during our qualitative analysis. If models possess information about GOAD and its contained vulnerabilities within their training set, we expect them to take shortcuts, i.e., use one of the well-known passwords from within its training set to skip initial access attacks. No occurrence of non-causal attack flows was detected within our log traces.

3.2.3 On using a realistic scenario instead of traditional benchmarks. Evaluating security tools and automated attack mechanisms on synthetic benchmarks has long been a common practice in cybersecurity research. However, as noted by Sommer and Paxson in “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection” [58], the limitations of synthetic environments can lead to an oversimplified understanding of adversarial behavior. Synthetic testbeds typically fail to capture the dynamic complexity and nuanced behaviors inherent in real-world networks, particularly in enterprise environments managed by Microsoft Active Directory. This motivates our decision to base our research on a realistic and complex testbed, GOAD.

One critical drawback of synthetic testbeds is their inability to replicate the subtleties of operations such as password spraying which is a commonly used attack vector. In realistic scenarios, an LLM may generate a password like “winter2022” that could lead to a successful login attempt, while even a minimal alteration, such as “winter2022!”, would result in an immediate error due to strict account lock-out policies. Synthetic environments often do not accurately model the consequence of such minute variations, and, if account lock-out mechanisms are disabled to accommodate the simulation, the intrinsic realism of the scenario is compromised. Without this dynamic interplay, synthetic benchmarks risk misrepresenting the true performance of automated attack strategies.

Furthermore, the nondeterministic nature of many exploits presents significant challenges in a synthetic setting. For example, while a system may indeed be vulnerable to a known exploit such as *EternalBlue*, the probability of a successful compromise is inherently low and subject to variability; in some cases, executing the exploit may even crash the target system. Such outcomes not only disrupt the current attack path but also impact subsequent attack vectors that would have been available in a dynamic, operational network. Synthetic testbeds, by their nature, often ignore these probabilistic effects and the cascading consequences of exploit-induced system instability.

Another essential aspect of real-life enterprise networks is the presence of abusable background activities—for instance, user interactions with network shares. These temporal patterns are critical when evaluating attacks like token-capture and lateral movement strategies (e.g., *pass-the-token* or *pass-the-hash*), where attackers lie on lookout for those patterns. In a synthetic benchmark, these time-based nuances are typically flattened or entirely absent further distorting their real-world applicability. To generate interpretable and actionable insights, we adopt a qualitative approach in conjunction with systematic pre-processing of quantitative data, as described in our experimental design (Section 3).

Our decision to use the GOAD testbed is driven by the need to embrace the complexity and dynamic behavior of real-world enterprise networks. By challenging our LLM-driven penetration testing prototype within a realistic simulation environment, we are better positioned to capture the intricate interactions, stochastic outcomes, and timing